CS779 Project: Neural Machine Translation for English–Bengali and English–Hindi

Kunal Chandra
240580
kunalc24@iitk.ac.in
Indian Institute of Technology Kanpur

Abstract

This report documents the development of a Neural Machine Translation (NMT) system for English-to-Bengali and English-to-Hindi translation. Development proceeded through four weekly iterations. I began with a vanilla GRU encoder–decoder as a baseline, then built a preprocessing pipeline using fastText embeddings, spaCy for English, and `indic-nlp-library` for the target scripts. From there the architecture moved to a bidirectional LSTM with Bahdanau attention, then to a vanilla Post-LN Transformer, and then to a stacked bidirectional GRU with global attention. For the testing phase I returned to the Transformer family with a Pre-Layer-Normalization design, weight tying, and GELU activations, which produced the best results. A byte-pair-encoding variant was also built during the testing phase; it segmented Bengali poorly and scored substantially worse despite a lower training loss, so the final submission used the word-level model.

1 Competition Result

Item	Value
CodaLab username	`k_240580`
Final rank	35
BLEU	0.162
chrF	0.417
ROUGE-L	0.445
Training-phase submissions	25
Testing-phase submissions	7

Table 1: Final competition standing.

Week	Submissions
Week 1	2
Week 2	4
Week 3	7
Week 4	12

Table 2: Training-phase submissions by week.

2 Problem Description

The competition required building an NMT system for English-to-Bengali and English-to-Hindi translation. Given a training set of sentence pairs and a set of test source sentences, the objective was to generate translations evaluated using BLEU, chrF++, and ROUGE-L. The principal challenges were:

- extracting and aligning the data from the supplied JSON files;
- handling large target vocabularies under GPU memory constraints;
- preventing overfitting on a comparatively small parallel corpus;
- keeping evaluation metrics reliable and comparable across weeks;
- using the available GPU efficiently, since training time was the binding constraint on how many architectures could be tried.

3 Data Analysis

3.1 Dataset Statistics

Pair	Train	Validation	Test	Avg. length
English–Bengali	68,849	9,836	19,672	16.8
English–Hindi	149,646	21,379	42,757	31.4

Table 3: Corpus sizes. Average sentence length was stable across splits (Bengali 16.8 / 16.87 / 16.9 and Hindi 31.4 / 31.45 / 31.5 for train / validation / test), indicating the splits were drawn from the same distribution.

Pair	Source vocabulary	Target vocabulary
English–Bengali	31,920	37,921
English–Hindi	33,366	31,680

Table 4: Vocabulary sizes after preprocessing, with a frequency-2 cutoff.

The Hindi corpus has roughly twice the average sentence length of the Bengali corpus, which matters more than the raw sentence count: attention cost grows quadratically in sequence length, so the Hindi pair was consistently the more expensive of the two to train.

3.2 Critical Data Issues

The raw text contained a large quantity of junk tokens, most visibly long digit sequences and alphanumeric tracking codes. These carry no translatable meaning but do inflate the vocabulary, so they were removed with regular expressions during preprocessing. Left in, they would have consumed embedding capacity on tokens the model can never usefully translate.

4 Model Description

4.1 Week 1: Vanilla GRU (Baseline)

Architecture: a simple encoder–decoder RNN using GRUs, with manual sequential decoding and teacher forcing during training.

Goal: produce a first submission and establish a minimal working baseline.

Key learning: this is the simplest recurrent setup. It trains quickly but compresses the entire source sentence into a single fixed-size vector, so it loses context on longer inputs.

4.2 Week 2: Bidirectional LSTM with Bahdanau Attention

1. **Model:** a bidirectional LSTM encoder–decoder with Bahdanau attention.
2. **Speed fix — dynamic padding:** this week was largely spent on latency. I introduced `pack_padded_sequence` and `pad_packed_sequence`, which ensure the recurrent layers compute hidden states only for non-padding tokens. This removed a substantial amount of wasted computation.
3. **Architecture upgrade:** the unidirectional GRU became a bidirectional LSTM, so the encoder sees both left and right context for every source token.
4. **Parameter changes:** embedding dimension 300 and hidden dimension 256. I set `embedding.weight.requires_grad = False` to freeze the pre-trained vectors, on the assumption that the pre-trained knowledge was worth preserving.
5. **Embeddings:** 300-dimensional fastText vectors were used as static embeddings.

4.3 Week 3: Vanilla Transformer

Architecture: the standard `nn.Transformer` in its Post-Layer-Normalization form, with 6 encoder and 6 decoder layers, trained under a `LambdaLR` schedule.

Key learnings: the latency problem was solved outright — unlike an RNN, the Transformer processes all positions in parallel, and multi-head attention handles long-range dependencies far better. Convergence, however, was slow and the Post-LN arrangement proved sensitive to the learning rate, which motivated the Pre-LN design adopted later.

4.4 Week 4: Stacked Bidirectional GRU with Attention

Architecture: a stacked bidirectional GRU encoder paired with a unidirectional GRU decoder using global attention.

Key learnings: this was the strongest recurrent model of the four weeks, but it confirmed that the RNN family is limited by its sequential nature and by the attention bottleneck. Adding global attention helped, yet the architecture could not match a Transformer's throughput on the same hardware budget.

4.5 Final Model: Pre-LN Transformer

The final architecture combines structural and regularization choices aimed at stability rather than raw capacity.

- **Depth:** 6-layer encoder and 6-layer decoder stacks.
- **Pre-LN:** layer normalization is applied *before* each attention and feed-forward block, which keeps gradient magnitudes well behaved at initialization and makes the warmup schedule usable [2].
- **Activation:** the Gaussian Error Linear Unit (GELU), standard in modern Transformer stacks and smoother than ReLU through a deep network [5].
- **Weight tying:** the target embedding matrix is shared with the final output projection [3]. This cut the parameter count appreciably and was the main defense against overfitting on a small corpus.

The implementation draws on three results. The multi-head attention mechanism and the overall encoder–decoder design come from *Attention Is All You Need* [1]. The Pre-LN arrangement follows *On Layer Normalization in the Transformer Architecture* [2], which was what finally allowed stable training at a high peak learning rate. The weight-tying technique is taken from *Tying Word Vectors and Word Classifiers* [3].

Decoding is greedy throughout. Beam search would likely have improved the scores, but it was too slow to fit inside the submission schedule.

During the testing phase I also built a pipeline using a BPE tokenizer. It produced poor segment boundaries on Bengali: training loss went down while BLEU went down with it, which is the clearest possible sign that the loss was measuring the wrong thing. That variant was dropped in favour of the word-level model.

After the final submission I continued training the same architecture for more epochs and it scored materially better, but the deadline had passed. Those predictions are included in the accompanying archive.

5 Experiments and Training

5.1 Data Preprocessing

The two sides of the corpus fail in different ways, so the pipeline is deliberately asymmetric.

5.1.1 Source Language (English)

1. **Normalization and contraction expansion:** raw text is stripped of URLs, HTML tags, and proprietary encodings. A custom routine expands contractions (`n't` → `not`, `'s` → `is`), so that informal and formal spellings collapse to one vocabulary entry.
2. **Tokenization:** spaCy's pipeline, with the parser and tagger disabled since only token boundaries are needed.
3. **Junk filtering:** pure punctuation, currency symbols, and alphanumeric junk such as tracking codes are discarded. This prevents vocabulary bloat.

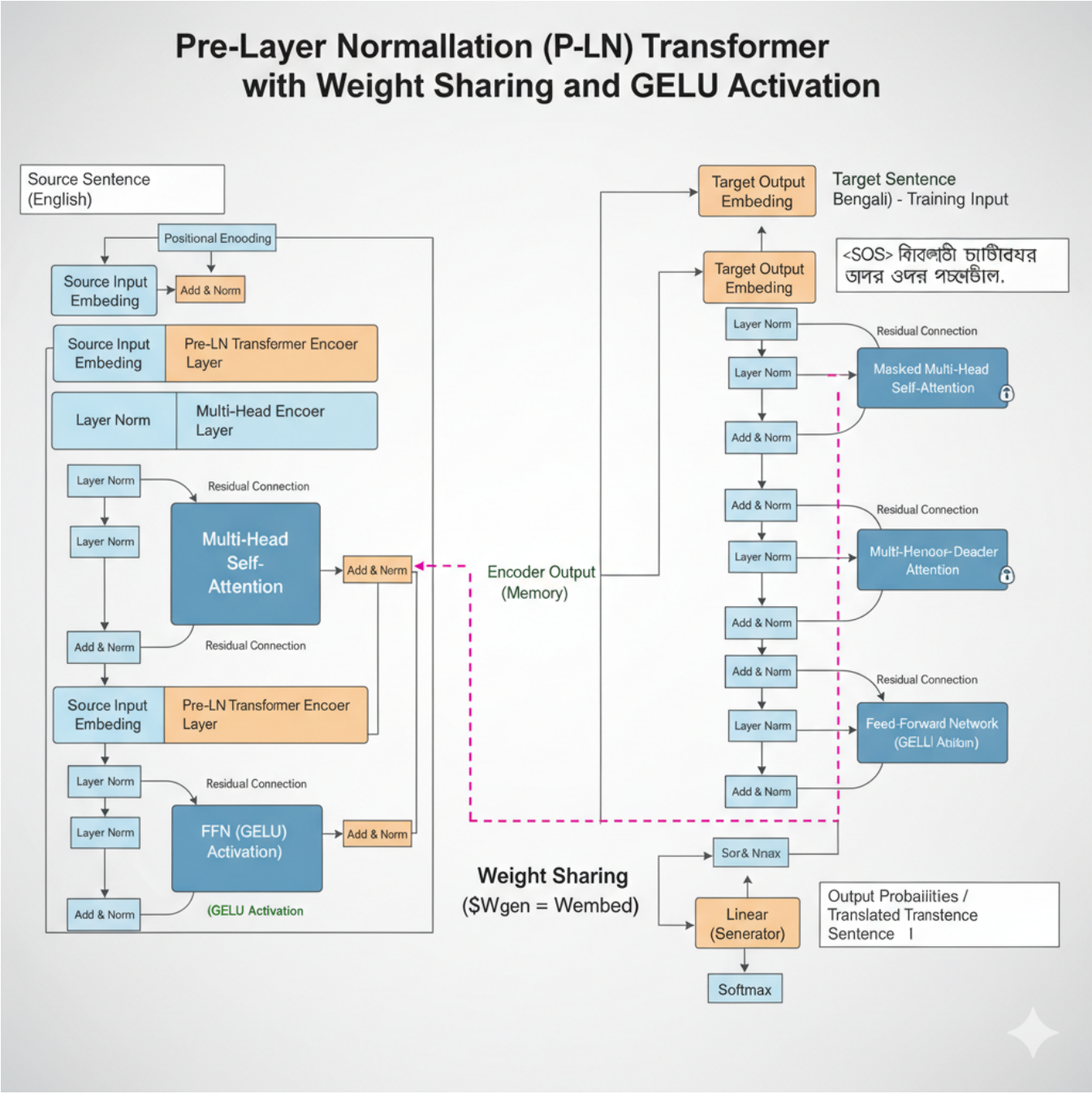

Figure 1: Final model architecture.

5.1.2 Target Languages (Bengali and Hindi)

1. **Noise removal:** as with English, URLs, HTML, and stray punctuation are removed.
2. **Character normalization:** `IndicNormalizerFactory` standardizes the script. This step matters more than it appears — Indic scripts admit several encodings of the same vowel or conjunct, and without normalization one word appears as several distinct vocabulary entries.
3. **Tokenization:** `indic_tokenize.trivial_tokenize` for token boundaries in the target script.
4. **Final filtering:** tokens are restricted to valid Bengali/Hindi characters and numerals.

The same pipeline was applied to the training, validation, and test splits, so that no distribution shift is introduced by preprocessing alone.

5.2 Training Configuration

5.2.1 Week 1: Vanilla GRU

Embedding size 128, Adam, learning rate 3×10^{-3}, `nn.NLLLoss`, batch size 64, greedy decoding, 8 epochs.

Justification: the small embedding minimized memory overhead and the high learning rate gave rapid initial convergence. The configuration existed only to establish a verifiable baseline and to expose the latency limits of sequential decoding.

5.2.2 Week 2: Bidirectional LSTM with Attention

Embedding size 300, hidden size 256, AdamW, learning rate 3×10^{-3}, `nn.CrossEntropyLoss`, batch size 32, greedy decoding, dropout 0.3, 8 epochs and then 22.

Justification: capacity was increased to make attention worthwhile, and dropout was introduced because the Week 1 model had shown no regularization at all. Out-of-memory errors and low GPU utilization were the main reasons for moving on.

5.2.3 Week 3: Vanilla Transformer

Model dimension 512, feed-forward dimension 2048, 8 attention heads, 6 encoder and 6 decoder layers, Adam, cross-entropy loss, `LambdaLR` with 4000 warmup steps.

Justification: these are the proportions of the original "base" Transformer, paired with the inverse-square-root warmup schedule the architecture was designed around.

5.2.4 Week 4: Stacked GRU

Embedding size 256, hidden dimension 512, 2 layers, dropout 0.3, cross-entropy loss.

Justification: this was the last recurrent configuration tried. The published evidence in favour of Transformers, combined with the throughput gap measured in Week 3, motivated the return to the Transformer family for the testing phase.

5.2.5 Testing Phase: Final Model

Hyperparameter	Value
Embedding / model dimension	512
Attention heads	8
Feed-forward dimension	1024
Encoder / decoder layers	6 / 6
Dropout	0.15
Maximum sequence length	55
Optimizer	AdamW ($\beta = 0.9/0.98$, weight decay 0.01)
Learning-rate schedule	Noam inverse square root, 4000 warmup steps
Loss	Cross-entropy, label smoothing 0.1
Precision	Mixed (fp16 autocast with gradient scaling)
Decoding	Greedy
Epochs	8 for the submitted model; 15 subsequently

Table 5: Final model configuration.

Raising the epoch count from 8 to 15 improved results noticeably, but that run finished after the submission deadline.

6 Results

6.1 Validation Set (Training Phase)

Model	chrF	ROUGE-L	BLEU
Vanilla GRU (baseline)	0.233	0.319	0.058
Bi-LSTM with attention	0.264	0.277	0.078
Vanilla Transformer (Post-LN)	0.280	0.340	0.073
Stacked GRU with global attention	0.326	0.362	0.102

Table 6: Validation performance by week.

The Week 3 Transformer is the interesting row: it beat the Bi-LSTM on chrF and ROUGE-L while scoring *lower* on BLEU. BLEU rewards exact n-gram matches, so a model producing broadly correct content with different phrasing can move the three metrics in opposite directions.

6.2 Test Set

Model	chrF	ROUGE-L	BLEU
Pre-LN Transformer with BPE	0.192	0.218	0.059
Pre-LN Transformer, word-level (submitted)	**0.417**	**0.445**	**0.162**

Table 7: Test performance.

The word-level Pre-LN Transformer was the strongest model on every metric. Pre-LN normalization and the AdamW optimizer gave stable optimization, while weight tying supplied the regularization needed to generalize from a small corpus. The BPE variant scored roughly a third as well despite reaching a lower training loss.

7 Error Analysis

The Pre-LN Transformer succeeded where the Post-LN version had not, because normalizing before each sub-block kept early training stable instead of diverging at the learning rates the warmup schedule reaches. The stacked Bi-GRU performed respectably once packing and padding were optimized, but lacked the capacity to compete.

The dominant error mode in the final model is out-of-vocabulary words. A frequency-2 cutoff leaves rare words — proper nouns above all — outside the vocabulary, where they are replaced by `<UNK>` and cannot be recovered. Word dropout during training would force the model to cope with missing tokens and should improve this; subword modelling would address it more directly, provided the segmentation is sound.

The most useful insight of the project was that a high-capacity Transformer was the wrong instrument for this corpus. The larger configurations did not have enough data to learn from and overfitted rather than generalized. A smaller, well-regularized Transformer outperformed them — capacity was never the binding constraint.

8 Conclusion

The project's outcome rested on diagnosing architectural instability and controlling overfitting, not on scaling the model up. My main finding is that for a structurally advanced architecture like the Pre-LN Transformer, stability and data robustness matter more than raw capacity. The volatility of the early RNN and Post-LN runs was resolved by adopting the Pre-LN architecture together with a simple, fixed optimization strategy.

Future directions. A correctly configured subword tokenizer such as BPE should help appreciably, since Bengali is morphologically rich and word-level vocabularies handle it badly — my attempt failed on segmentation quality, not on the premise. Beam search decoding would likely raise all three metrics at some cost in inference time. Finally, I used a single architecture for both language pairs throughout; given that Hindi sentences average almost twice the length of Bengali ones, tuning each pair separately is likely to pay off.

References

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). *Attention Is All You Need.* Advances in Neural Information Processing Systems 30. https://arxiv.org/abs/1706.03762

2. Wang, Q., Li, B., Xiao, T., Zhu, J., Li, C., Wong, D. F., and Chao, L. S. (2019). *Learning Deep Transformer Models for Machine Translation.* ACL 2019. https://arxiv.org/abs/1906.01787

3. Press, O., and Wolf, L. (2017). *Using the Output Embedding to Improve Language Models.* EACL 2017. https://arxiv.org/abs/1608.05859

4. Loshchilov, I., and Hutter, F. (2019). *Decoupled Weight Decay Regularization.* ICLR 2019. Establishes that L_2 regularization and weight decay are not equivalent for adaptive optimizers, which is why AdamW is used here. https://arxiv.org/abs/1711.05101

5. Hendrycks, D., and Gimpel, K. (2016). *Gaussian Error Linear Units (GELUs).* https://arxiv.org/abs/1606.08415